

```

<html>
<head>
<link rel="stylesheet" href="https://pyscript.net/alpha/
pyscript.css" />
<script defer src="https://pyscript.net/alpha/pyscript.js"></
script>
</head>
<py-title>PyScript Playground</py-title>

<body>
<py-button label="Click me" styles="btn big">
def on_click(event):
print(event)
</py-button>

<br>
<br>
<py-inputbox>
def on_keypress(event):
print(event)
</py-inputbox>

<br>
<br>
<py-box widths="2/3;1/6;1/6">
<div>Wide Column</div>
<div>Narrow Column</div>
<div>Narrow Column</div>
</py-box>

</body>
</html>

```



Figure 6: PyScript playground

JavaScript vs PyScript

So, do we have a new language in PyScript that will replace JavaScript? That's a million-dollar question.

JavaScript is a high-level programming language and one of the core technologies of the World Wide Web. And it

is used as a client-side programming language by 97.8 per cent of all websites.

However, Python is not far behind in the popularity race and is one of the most liked languages in the world. So definitely, but not immediately, there will be a user space shift from JavaScript to PyScript. This is specifically in the data science world, as it helps build pages with plots easily. Nonetheless, performance is where PyScript lags far behind, but this is just the beginning and I am confident the language will evolve.

I am sure we will continue to use loads of JavaScript. It's not easy or necessary to replace it yet. Having said that, PyScript or Python and other language-based browser scripting will make their own user space too. PyScript is bound to grow using WASM technology.

In the end, let's just recap a few important tags:

- `py-script` tag is where you write your Python code that gets executed within the web page.
- `py-env` tag defines the Python packages needed to run your Python code.
- `py-repl` tag creates a REPL (read-eval-print loop) component that evaluates the code users enter and displays the results.
- `py-config` tag sets and configures general metadata about your PyScript application in YAML format.

In my opinion, PyScript is good for proof-of-concepts, but I would be a little hesitant to use it for business for some more time. Performance and a few bugs are two big reasons at present. Nonetheless, this is a very good attempt based on a vision of the future. **END** 🐧

References

- <https://pyscript.net>
- <https://anaconda.cloud/pyscript-python-in-the-browser>
- <https://github.com/pyscript/pyscript>
- <https://github.com/pyscript/pyscript/tree/main/examples>
- <https://groups.google.com/g/brython/c/QWDBKCZnIEM?pli=1>
- https://brython.info/static_doc/en/faq.html
- <https://pyscript.net/examples/> (demo for all the examples from the github)
- <https://www.infoworld.com/article/3661628/get-started-with-pyscript-the-in-browser-python-by-anaconda.html>
- <https://realpython.com/pyscript-python-in-browser/>
- https://www.youtube.com/watch?v=t_R-nxHMWEg [MUST WATCH]

By: Thangaselvi Arichandrapandian

The author is a lifelong learner influenced by the quote:
 "The more I learn, the more I realise how much I don't know."
 -Albert Einstein